

FairPrice / BirrWise

A Community Price Tracker

Group 5 – Mobile Application Development

Addis Ababa University, Semester 2, 2025/2026

Submission Date: January 4, 2026

Group Information

The following table outlines the roles and student IDs for each member of Group 5:

Role	Full Name	Student ID
Team Leader	Yasin Jama	ATE/4368/15
Developer	Getamesay Hailemichael	ATE/5152/13
UI/UX Designer	Sisay Leykun	ATE/0493/15
Tester	Tamrat Arage	ATE/8888/15

Project Overview

This section provides a high-level overview of the FairPrice / BirrWise project, outlining its key characteristics and target audience.

Detail	Value
Project Title	FairPrice / BirrWise – Community Price Tracker
Category	FinTech / Utility
Platform	Android & iOS (Flutter cross-platform)
Target Users	Ethiopian consumers, price-conscious shoppers

Problem Statement & Solution

This section details the core challenges consumers face regarding market prices and introduces FairPrice / BirrWise as the proposed solution.

Problem

Prices vary significantly between shops, making it hard for consumers to know fair market prices. People are frequently overcharged due to lack of price transparency and limited ability to compare prices across different locations.

Solution

A crowdsourced price transparency system that empowers consumers with real-time market data. FairPrice allows users to report, verify, and compare product prices from nearby shops, view price trends, set alerts, and earn community points.

Key Features

FairPrice / BirrWise offers a comprehensive suite of features designed to empower consumers with market insights and foster community participation:

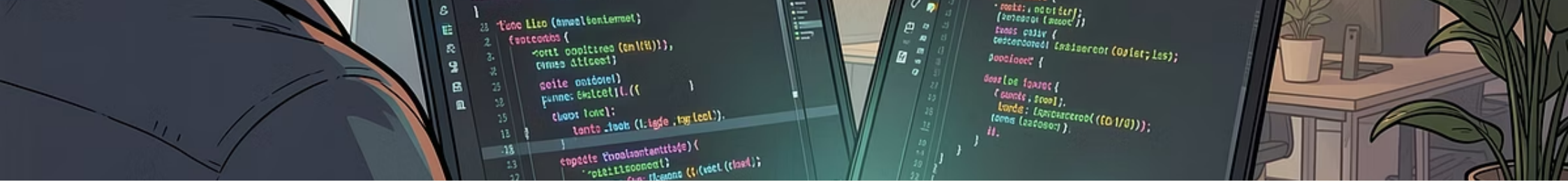
1. Product Price Search - Search by name or barcode; filter by category & price range
2. Crowdsourced Reporting - Report prices with shop details and optional notes
3. Price Comparison - Compare same product across multiple shops
4. Location-Based Map - GPS-powered map showing nearby shops with distance & status
5. Price History Charts - Visual line charts for 7/14/30 days
6. Watchlist & Push Alerts - Set target price; receive FCM notification when price drops
7. Community Leaderboard - Earn points for reporting/verifying; top contributors ranked
8. Barcode Scanner - Scan product barcode for instant lookup or reporting
9. Admin Dashboard - Real-time stats, user management, approve admin requests
10. Localization - Full Amharic and English support (AR/EN)
11. Dark Mode & Shopping List - Theme switching and persistent grocery checklist



Technology Stack

FairPrice / BirrWise leverages a modern and robust technology stack to ensure performance, scalability, and an intuitive user experience. The following table details the key technologies and tools used in its development:

Category	Technology / Tool
Framework	Flutter (Dart) – cross-platform
Backend / Database	Firebase Firestore (NoSQL, real-time)
Authentication	Firebase Auth (email/password)
Push Notifications	Firebase Cloud Messaging (FCM)
Storage	Firebase Storage (profile photos)
Maps	flutter_map + OpenStreetMap + Geolocator
Charts	fl_chart
Barcode Scanning	mobile_scanner
Image Caching	cached_network_image
Time Formatting	timeago
Localization	flutter_localizations + intl
State Management	Provider + Firestore streams
Local Storage	SharedPreferences
Version Control	Git & GitHub
UI/UX Design	Figma
Testing	Manual + Flutter test



Key Code Snippets (part1)

This section highlights the core code implementations behind FairPrice / BirrWise and shows how the app handles data, real-time updates, and authentication.

8.1 Data Models: Product Class (models.dart)

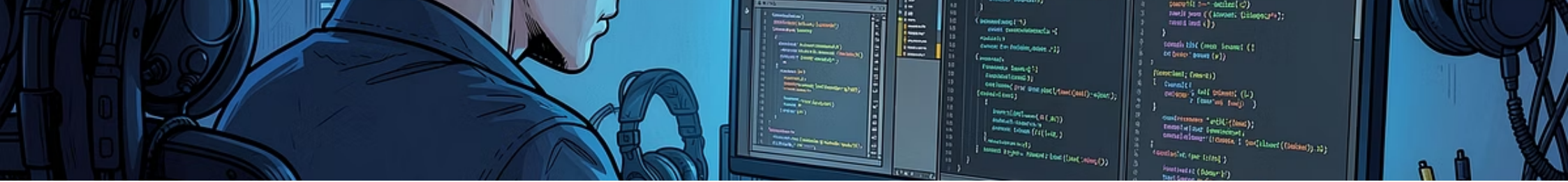
The `Product` class defines product data such as `name`, `price`, `shopName`, and `timestamp`. Its `fromFirestore()` constructor converts Firestore snapshots into typed objects, while `toJson()` supports writing data back to the database.

```
class Product {
  final String? id;
  final String name;
  final double price;
  final String shopName;
  final DateTime timestamp;

  Product({
    this.id,
    required this.name,
    required this.price,
    required this.shopName,
    required this.timestamp,
  });

  factory Product.fromFirestore(String id, Map<String, dynamic> data) {
    return Product(
      id: id,
      name: data['name'] ?? '',
      price: (data['price'] ?? 0).toDouble(),
      shopName: data['shopName'] ?? '',
      timestamp: (data['timestamp'] as Timestamp?).toDate() ?? DateTime.now(),
    );
  }

  Map<String, dynamic> toJson() {
    return {
      'name': name,
      'price': price,
      'shopName': shopName,
      'timestamp': Timestamp.fromDate(timestamp),
    };
  }
}
```



Key Code Snippets (Part 2)

Continuing our deep dive into the FairPrice / BirrWise codebase, this section reveals further critical implementations. We explore how the application manages its global state and integrates a barcode scanner for quick product lookups. These snippets showcase the robustness and modularity achieved through Flutter's reactive framework and well-structured service layers.

8.4 App Provider: State Management (app_provider.dart)

The `AppProvider` centralizes global state management using Flutter's `ChangeNotifier`. It allows for efficient updates across the application, handling user sessions, theme preferences, and localization settings. This approach ensures that UI components react seamlessly to changes, providing a consistent and personalized user experience.

```
import 'package:flutter/material.dart';
import 'package:birrwise/models/user.dart';

class AppProvider with ChangeNotifier {
  User? _currentUser;
  ThemeMode _themeMode = ThemeMode.system;
  Locale _locale = const Locale('en');

  User? get currentUser => _currentUser;
  ThemeMode get themeMode => _themeMode;
  Locale get locale => _locale;

  void setUser(User? user) {
    _currentUser = user;
    notifyListeners();
  }

  void setThemeMode(ThemeMode mode) {
    _themeMode = mode;
    notifyListeners();
  }

  void setLocale(Locale newLocale) {
    _locale = newLocale;
    notifyListeners();
  }
}
```

8.5 Barcode Scanner Integration: `_onDetect` Method (barcode_scanner_view.dart)

The `_onDetect` method is the core of the barcode scanning functionality. Triggered upon a successful scan, it processes the barcode data (EAN-13 or UPC-A), performs a lookup for the product in Firestore, or navigates to a product reporting screen if the item is new. This real-time processing provides instant feedback to the user.

```
import 'package:mobile_scanner/mobile_scanner.dart';
import 'package:flutter/material.dart';

void _onDetect(BarcodeCapture capture, BuildContext context) {
  final List barcodes = capture.barcodes;
  for (final barcode in barcodes) {
    debugPrint('Barcode found! Data: ${barcode.rawValue}');
    final String? barcodeData = barcode.rawValue;
    if (barcodeData != null) {
```




Key Code Snippets (Part 3)

This section concludes our detailed look at the core code snippets, focusing on essential administrative controls and robust localization features. These implementations are vital for maintaining application integrity, managing user roles, and delivering a culturally relevant experience to the Ethiopian user base, specifically supporting Amharic alongside English.

8.8 Admin Approval Flow

Administrative functionalities are crucial for content moderation and user management. These methods handle requests from regular users seeking elevated admin privileges. The `approveAdminRequest()` method promotes a user to an admin role and removes their request, while `rejectAdminRequest()` simply dismisses the request without granting access. This batch operation ensures atomicity, preventing partial updates in the database.

```
// lib/services/admin_service.dart

import 'package:cloud_firestore/cloud_firestore.dart';

class AdminService {
  final FirebaseFirestore _db = FirebaseFirestore.instance;

  Future<void> approveAdminRequest(String userId, String requestId) async {
    WriteBatch batch = _db.batch();

    // 1. Update user role
    DocumentReference userRef = _db.collection('users').doc(userId);
    batch.update(userRef, {'isAdmin': true});

    // 2. Remove the admin request
    DocumentReference requestRef = _db.collection('adminRequests').doc(requestId);
    batch.delete(requestRef);

    await batch.commit();
    print('Admin request approved for user $userId');
  }

  Future<void> rejectAdminRequest(String requestId) async {
    // Simply remove the request
    await _db.collection('adminRequests').doc(requestId).delete();
    print('Admin request rejected: $requestId');
  }
}
```

8.9 Localization Setup

To cater to its primary user base, FairPrice / BirrWise supports both English and Amharic. This Flutter `MaterialApp` configuration demonstrates the setup for internationalization, integrating custom localization delegates and defining the supported locales to ensure the application text adapts dynamically to the user's preferred language. The `localeResolutionCallback` prioritizes user's device settings but defaults to English if no match is found among supported languages.

```
// lib/main.dart (excerpt)

import 'package:flutter/material.dart';
import 'package:flutter_localizations/flutter_localizations.dart';
import 'package:fairprice/l10n/app_localizations.dart'; // Custom generated localizations

class MyApp extends StatelessWidget {
```



Key Code Snippets (Part 4)

8.10 Reusable UI Component: ProductListTile

This snippet introduces a fundamental UI component, `ProductListTile`, designed to display individual product information consistently across the application. By centralizing the UI logic for a product listing, this reusable widget significantly reduces code duplication, enforces a uniform visual presentation, and streamlines future design updates or enhancements. It adheres to best practices in UI development, promoting modularity and maintainability.

```
// lib/widgets/product_list_tile.dart

import 'package:flutter/material.dart';

class ProductListTile extends StatelessWidget {
  final String productName;
  final String productPrice;
  final String? imageUrl;
  final VoidCallback? onTap;

  const ProductListTile({
    Key? key,
    required this.productName,
    required this.productPrice,
    this.imageUrl,
    this.onTap,
  }) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Card(
      margin: const EdgeInsets.symmetric(horizontal: 16.0, vertical: 8.0),
      elevation: 2.0,
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(10.0)),
      child: ListTile(
        contentPadding: const EdgeInsets.all(12.0),
        leading: imageUrl != null && imageUrl!.isNotEmpty
          ? CircleAvatar(
              backgroundImage: NetworkImage(imageUrl!),
              radius: 25,
            )
          : const CircleAvatar(
              child: Icon(Icons.shopping_bag_outlined),
              radius: 25,
            ),
        title: Text(
          productName,
          style: const TextStyle(fontWeight: FontWeight.bold, fontSize: 16.0),
        ),
        subtitle: Text(
          productPrice,
          style: TextStyle(color: Colors.grey[700], fontSize: 14.0),
        ),
        trailing: const Icon(Icons.arrow_forward_ios, size: 16.0, color: Colors.grey),
        onTap: onTap,
      ),
    );
  }
}
```

System Architecture

FairPrice / BirrWise is built upon a robust, scalable, and real-time architecture, primarily leveraging Google's Firebase ecosystem and Flutter for cross-platform mobile development. This design ensures efficient data handling, seamless user experiences, and straightforward maintenance.



Data Flow

The system is designed for real-time data synchronization and robust access control:

- Firestore `snapshot()` provide real-time UI updates, ensuring users always see the most current price information and community contributions without manual refreshing.
- Offline persistence is enabled through a combination of the Firestore cache for database queries and `SharedPreferences` for local user settings and preferences, allowing the app to function even without an active internet connection.
- Role-based access controls regulate user privileges: **user** (standard participant), **pending_admin** (awaiting administrator approval), and **admin** (full system management and moderation capabilities).

Database Schema

The FairPrice / BirrWise application relies on a flexible and scalable NoSQL database schema implemented in Firebase Firestore. Below is a clean code-style overview of the core collections used for real-time data and user interactions:

users

- uid
- displayName
- email
- role
- banned
- reportCount
- verifyCount
- points
- fcmToken
- photoUrl
- location

products

- name
- category
- unit
- emoji
- minPrice
- maxPrice
- avgPrice
- changePercent
- reportCount
- priceHistory
- barcode

shops

- name
- area
- distanceKm
- rating
- isOpen
- closingTime
- reportCount
- lat
- lng
- prices

Database Schema (Part 2)

Continuing our exploration of the FairPrice / BirrWise Firestore structure, this section details the critical collections responsible for managing user-contributed data and administrative workflows: **reports** and **admin_requests**. These collections are pivotal for maintaining data integrity, fostering community participation, and ensuring proper moderation within the application ecosystem.

Reports Collection

The reports collection is central to community-driven data validation and content moderation. Whenever a user submits new price information, flags inaccurate data, or provides additional context, a document is created here. This system allows for collective verification and ensures that product price data remains current and reliable. Key fields track the reported item, the reporter's identity, the reported price, and a mechanism for community verification.

```
// Collection: reports
{
  productId: "string",    // ID of the product being reported
  productName: "string",  // Name of the product (for display and quick lookup)
  shopId: "string",       // ID of the shop where the report was made
  shopName: "string",     // Name of the shop (for display)
  price: "number",        // The reported price of the product
  reporterUid: "string",   // User ID of the individual who submitted the report
  reporterName: "string",  // Display name of the reporter
  reportedAt: "timestamp", // Timestamp when the report was submitted
  verifyCount: "number",   // Count of users who verified the report's accuracy
  noCount: "number",      // Count of users who marked the report as inaccurate
  type: "string",         // Type of report (e.g., "price_update", "inaccurate_info", "shop_closed")
  note: "string"          // Optional additional notes or comments from the reporter
}
```

Admin Requests Collection

The admin_requests collection facilitates the process for users to apply for elevated administrative privileges. This structured approach ensures that all requests are formally logged, reviewed, and managed by existing administrators. It supports the application's growth by allowing community members to contribute to moderation and management in a controlled manner, enhancing the overall integrity and trustworthiness of the platform.

```
// Collection: admin_requests
{
  uid: "string",          // User ID of the applicant
  displayName: "string",  // Display name of the applicant
  email: "string",        // Email address of the applicant
  reason: "string",       // Detailed reason for requesting admin access
  organization: "string", // Optional: Organization the applicant belongs to
  status: "string",       // Current status of the request (e.g., "pending", "approved", "rejected")
  requestedAt: "timestamp", // Timestamp when the request was submitted
  reviewedAt: "timestamp",  // Timestamp when the request was reviewed
  reviewedBy: "string"     // User ID of the admin who reviewed the request
}
```

These collections are crucial for the dynamic and interactive nature of FairPrice / BirrWise, empowering users while providing the necessary tools for platform governance and data quality assurance. They underpin the collaborative spirit of the application, ensuring a robust and trustworthy information network.

Testing Summary

The development process for FairPrice / BirrWise included a comprehensive testing strategy to ensure the application's reliability, functionality, and user experience. The following table summarizes the different types of testing performed, who was responsible for them, and their status:

Test Type	Performed By	Status
Unit Testing (models, services)	Developer + Tester	✔ Pass
Widget Testing (UI components)	Tester	✔ Pass
Integration Testing (user flows)	Tester	✔ Pass
Manual Testing (Android/iOS)	Tester	✔ Pass
Firebase Rules Testing	Team Leader	✔ Pass
User Acceptance Testing (5 real users)	Team Leader	✔ Pass

Deployment

The FairPrice / BirrWise application employs a streamlined deployment process to ensure efficient distribution and controlled access to its codebase.

Android APK

The Android Application Package (APK) for release builds is generated using the command: `flutter build apk --release`.

Internal Testing

For internal testing and distribution to designated testers, **Firebase App Distribution** is utilized, providing a robust platform for beta testing.

Source Code

The application's source code is maintained in a **GitHub repository**, which is kept private and shared exclusively with the instructor to ensure controlled access and collaboration.

Challenges & Solutions

During the development of FairPrice / BirrWise, several technical challenges were encountered and successfully addressed, leading to a more robust and stable application. Below is a summary of these challenges and their implemented solutions.

Challenge	Solution
Add product dialog causing dark screen	Used global ScaffoldMessengerKey + proper context handling
Admin close button → empty screen	Redirect to user home instead of Navigator.pop
Real-time updates	Firestore streams with Provider
FCM token registration	Save token on login & token refresh

Future Enhancements

Building on the current foundation, the FairPrice / BirrWise application has several exciting future enhancements planned to expand its functionality and user engagement, outlined below as a bulleted list of planned improvements:

- Payment integration (Chapa, Telebirr)
- AI-based price prediction
- Shop loyalty programs
- Web admin dashboard
- User chat for price negotiation

Group Declaration

We, the undersigned, confirm that all group members have actively participated, the project is original work of Group 5, and we have followed all project guidelines.

Role	Name	Signature & Date
Team Leader	Yasin Jama	_____ / / _
Developer	Getamesay Hailemichael	_____ / / _
UI/UX Designer	Sisay Leykun	_____ / / _
Tester	Tamrat Arage	_____ / / _